

# Gas Profiler Module Report

## Overview

As part of Milestone 2 of the Plutus Debugger VSCode Plugin, I implemented a Gas Profiler module to help developers better understand how their Plutus validators consume execution resources during simulation. The goal of this feature is to make validator optimization easier by showing CPU steps, memory usage, execution timing, and the most expensive operations directly inside VSCode. The profiler currently supports Plutus V2 and V3 style execution metrics.

---

## How the Profiler Works

When a validator is simulated inside the extension, the execution result returns information about CPU steps, memory usage, execution timing, script hash, and operation-level execution details. The profiler reads this execution data and converts it into a more visual format that is easier to understand during development. Instead of only showing raw JSON output, the extension also generates memory usage indicators, CPU usage bars, operation heat visualization, and top costly operation rankings. This makes it easier to quickly identify expensive parts of validator execution.

---

## Identifying High-Cost Operations

The profiler identifies costly operations by comparing the execution-unit usage of each validator operation returned during simulation. Operations consuming higher CPU or memory resources are ranked and displayed inside the “Top 3 Costly Operations” section. During testing, operations such as `verifyEd25519Signature`, `serialisePlutusData`, and `equalsData` frequently appeared as expensive execution paths depending on validator complexity. The heat bars shown in the interface are calculated relative to the total execution cost of the validator. This gives developers a quick visual indication of where optimization may be needed.

---

## Example Execution Metrics

Example simulation output collected by the profiler: • Memory Usage: 800 • CPU Steps: 112100 • Execution Time: approximately 70 milliseconds The extension also displays the percentage contribution of costly operations to overall execution cost.

---

## Why This Feature Is Useful

One of the main difficulties while developing Plutus validators is understanding why certain scripts become expensive during execution. Usually developers only see final execution-unit numbers, which makes optimization difficult. The Gas Profiler improves this workflow by showing where execution cost is coming from, which operations are expensive,

and how validator complexity changes execution behavior. Because everything runs directly inside VSCode, developers can test and profile validators without switching between multiple tools.

---

## Conclusion

The Gas Profiler module adds practical execution analysis capabilities to the Plutus Debugger VSCode Plugin. It helps developers better understand validator execution behavior and makes optimization more accessible during smart contract development on Cardano.

---